

Can an AI proof loop replace model checking?

A measured comparison of TLC against machine-closed Lean proofs
with wall-clock and dollar costs

Gavin Mendel-Gleason

Maren Roos

September 27, 2026

Abstract

Model checking a TLA^+ specification with TLC is push-button and exhaustive, but only over a *bounded* instance: it answers “no violation up to N ”. A proof answers “for all N ”, and until recently the cost of obtaining one was human-weeks. We ask whether an AI proof loop — a language model driving Lean 4 through a shell until a mechanically checked closure oracle accepts the artifact — can close that gap, and at what cost.

Across four concurrent algorithms (token-ring mutual exclusion, Lamport’s Bakery, LCR leader election and EWD998 termination detection) the loop closed every task’s general theorem, each with a negative control demonstrating that the rig detects a weakened model. **The headline claim is supported on 4 of 4 tasks: the loop proves the general theorem inside a 2 h / \$50 budget, by two to three orders of magnitude, where TLC cannot answer at any budget.** The same-claim cost comparison is 2–2 and turns on where each instance falls relative to its own crossover rather than on the methods: TLC is faster on two tasks, the loop on two, and cost totals favour the loop on three of four. Every closure cost under a millicent of a dollar, and every proof total — general theorem plus corollary — under 1.5 millidollars.

We report the method, the four tasks, the measured matrix, and at length what the result does *not* establish.

1 Introduction

TLC decides a bounded instance by enumeration. That is the right instrument for a design under development, and its limitation is structural rather than a matter of speed: it cannot establish an invariance for arbitrary N , and its cost grows exponentially in the instance size. Theorem provers answer the general question, but the published human effort is substantial. The TLA^+ Trifecta paper [1] reports **one person-day** for an invariance proof of EWD998, and the IJCAR 2010 TLAPS papers [2] report proof-line counts in the hundreds with no time figure at all.

Three developments make the question worth asking now: reasoning models that can plan proof steps, Lean 4 [3] with Mathlib [4] as a target whose kernel is small and whose libraries are broad, and — the part we emphasise — the ability to *verify* a candidate proof mechanically rather than trust the model that produced it. Recent work measures how often such models can prove benchmark problems [9, 10, 11]. We measure something different: what a *closed* theorem costs in wall-clock and dollars, against the model checker it would replace, on the same specification.

Our question is therefore: *can a loop close general theorems for concurrent systems, at what wall-clock and dollar cost, and how does that compare with what TLC costs on the same claim?*

2 Method

2.1 Two routes, two tiers

Route A is TLC enumerating a TLA^+ specification at a bounded instance. **Route B** is a Lean 4 theorem closed by an AI proof loop.

Each task has two Route B tiers. **Tier 2** is the general theorem: the property for arbitrary N . **Tier 1** is that theorem’s corollary, instantiated at the calibrated instance N_0 . Only tier 1 is comparable with Route A, because only tier 1 proves the same claim.

N_0 is fixed by a stated rule: **the largest N whose TLC run completes inside the 2 h per-run cap**. That definition matters in §5.

2.2 The closure criterion

A *closed* verdict comes from three checks run by code the candidate cannot reach:

1. **Integrity** — the candidate is byte-identical to the seed up to and including the theorem’s $:=$. This pins every definition and the statement; only the proof body is free.
2. **Elaboration** — the candidate is handed to `lean` directly, with no `lake` in the invocation. The candidate has a shell and can reach the package’s lakefile, so a lake-built environment would be *the candidate’s*, not the harness’s.
3. **Closure** — a prebuilt checker imports the elaborated `olean` and reads the declaration’s axiom set out of Lean’s *API*, against `{propext, Classical.choice, Quot.sound}`.

The third is load-bearing and its implementation matters. `#print axioms` is *syntax*, syntax is an environment extension a candidate can install, and it crosses imports — in testing, a candidate rewrote the command into text of its own and the real report never ran. An API query cannot be reached that way.

Around those three: the bytes checked are one immutable snapshot taken once per round and never re-read, so the verdict, the result row and the durable closure copy all derive from the same bytes; the elaboration environment is captured before the session opens; and the seed’s digest is re-checked on every exit path.

2.3 Why a rate, not a run

Closure is stochastic. The unit of evidence is therefore a **cell**: $R \geq 5$ runs sharing one seed digest, never pooled across configurations. The complement is the **negative control** — the loop is run against the task’s *mutant*, a weakened model, and must *fail* to close it. Without that, a working prover and a broken rig are indistinguishable.

2.4 Measurement

We record wall-clock, peak RSS, dollar cost from provider usage at list price, and distinct states enumerated. Route A runs at its best measured configuration with the row naming it, and worker count is reported because it moves wall-clock but not state counts.

Task	Property	Source
token-ring	mutual exclusion	our bootstrap model
Bakery	mutual exclusion	Lamport [5]
LCR	at most one leader	Le Lann [7], Chang & Roberts [8]
EWD998	termination detection (invariance)	Dijkstra [6]; imported with the proofs of [1]

Table 1: The four tasks. EWD998 is externally calibrated: both the specification and the published TLAPS proofs are imported at pinned provenance, and the human-effort figures attach to it.

	token-ring	Bakery	LCR	EWD998
Route A N_0	23	9	10	3
Route A wall-clock	837.9 s (8 workers)	4,847.7 s	2.7 s	36.4 s
Route A cost	\$0.04655	\$0.26932	\$0.00015	\$0.00202
Route A distinct states	289,406,976	238,803,200	177,147	1,520,618
tier 2 (general N)	closed 104.8 s	closed 444.4 s	closed 216.0 s	closed 1009.5 s
tier-2 cost	\$0.00033	\$0.00062	\$0.00044	\$0.00081
tier 1 (corollary)	closed 38.5 s	closed 76.6 s	closed 101.0 s	closed 101.2 s
tier-1 cost	\$0.00023	\$0.00048	\$0.00042	\$0.00062
negative control	TLC violation	fail_to_close	fail_to_close	fail_to_close

Table 2: The measured matrix. Tier-2 figures are cell medians ($n = 6, 5, 5, 1$ respectively); ranges are wide — token-ring’s spans 62.9–1021.8 s — so no figure is quoted without its range. Route A at N_0 is a single deterministic run except EWD998, whose six runs give the median shown.

3 The task set

4 Results

4.1 The measured matrix

4.2 Same-claim cost, counted honestly

A corollary is the trivial instantiation of its theorem, so the corollary’s cost alone answers “what does instantiation cost *given* the theorem”. The question asked is what *settling the claim* costs, which is theorem plus corollary:

Task	Route A	theorem	corollary	proof total	K	favours
token-ring ($N=23$)	837.938 s	104.8 s	38.5 s	143.4 s	0.171	proof, 5.8×
Bakery ($N=9$)	4,847.749 s	444.4 s	76.6 s	521.0 s	0.107	proof, 9.3×
EWD998 ($N=3$)	36.4 s	1009.5 s	101.2 s	1,110.8 s	30.5	TLC, 30.5×
LCR ($N=10$)	2.670 s	216.0 s	101.0 s	317.0 s	118.7	TLC, 118.7×

Table 3: $K = \text{proof} \div \text{TLC}$ on the same claim. **The split is 2–2.** On cost the totals are kinder to the proof — cheaper on three of four; only LCR favours TLC on both resources.

4.3 Against published human effort

Only EWD998 has a published person-day figure, so the human comparison is one number against one number: a person-day against **143–1111 seconds** of loop time and **under 1.5 millidollars**. The IJCAR papers report lines, and we do not convert lines into days.

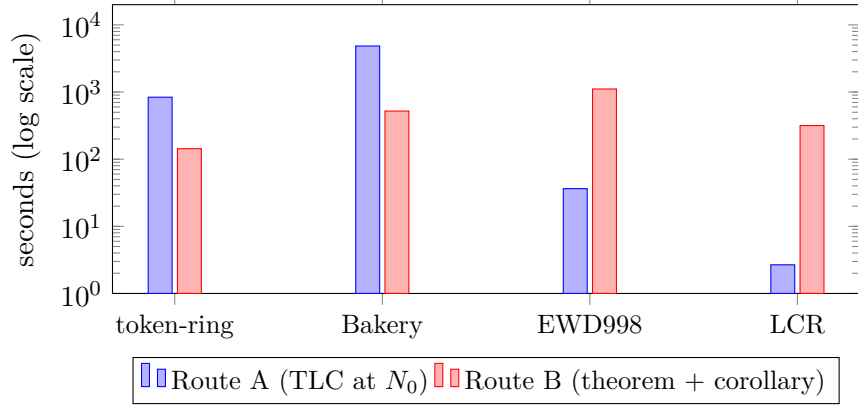


Figure 1: The same claim, both routes, log scale. The two routes cross between Bakery and EWD998. The crossover is a property of each task’s instance relative to its own state curve, not of the methods — which is why the ratios split rather than agreeing.

Our task	Human comparator	Reported
ewd998	TLAPS invariance	1 person-day , ~230 lines
ewd998	TLAPS safety + refinement	0.5 person-day , ~110 lines
ewd998	TLAPS liveness	<1 person-day , 245 lines
ijcar2010-*	TLAPS proofs	lines only — no time reported
token-ring, Bakery, LCR	none published	—

Table 4: Cited data, never a measured arm: the human figures are imported with quotes and pinned provenance, marked `machine_checked: false`, and never pooled with either route.

4.4 The verdict, against each reading of “replacement”

Reading	Result
General theorem inside budget B	Supported, 4 of 4. Worst case 1009.5s = 14.0% of the 2h cap, and \$0.00081 = 0.0016% of the \$50 cap. TLC’s coverage of this tier is 0 of 4 at any budget.
Cost parity at the bounded tier (K)	2–2: $K = 0.171 / 0.107 / 30.5 / 118.7$.
Coverage under a fixed budget	Counts equal at 4 of 4, claims not equal: the loop settles four <i>general</i> theorems; TLC settles four <i>bounded</i> instances and zero of the general tier.

Table 5: Protocol readings of what “replacing model checking with proof” might mean, each reported rather than one chosen for the reader.

5 What we learned that we did not set out to learn

A harness flag cost us a wrong calibration, and how we found it is the more useful result. EWD998’s instance search kept failing at $N = 4$ –6 while other tasks completed at far larger state counts. We formed four hypotheses; three died on evidence.

- *The failure is deterministic.* False. The two $N=4$ attempts died at **43,634,504** and **75,753,775** distinct states — different counts.

- *Our flag is the whole cause.* Half true, and the surviving half mattered. Every failing invocation carried TLC’s `-cleanup`, which deletes the state pool mid-enumeration. Without it, $N=4$ **completes** — 248,006,200 distinct states, depth 104, 2 h 36 min, exit 0 — where the flagged runs died. But Bakery completed at 238,803,200 states and token-ring at 289,406,976 *with* the flag in place, so the flag is **necessary but not sufficient**: it kills this specification’s runs and not others’.
- *A deep frontier breaks the disk queue.* False. Bakery held **234,890,583** states on queue and ran to its cap without error.

The discipline that separated them is worth stating plainly: **the surviving hypothesis was the one checked against the other tasks’ logs before being believed**. The three that died were each asserted from a single observation — one run compared against its own log, one failure mode generalised from one task, one intuition about frontier depth. In a measurement programme, a claim that has not been checked against an independent arm is a hypothesis, not a result.

And the calibration value did not change. $N_0 = 3$ stands — but for a different reason than it was first set. The flag made it an artifact of our invocation; the 2 h cap makes it a property of the stated rule, since $N=4$ needs 2 h 36 min. The number is the same and the ground is now sound. We report it that way rather than as a confirmation.

Rigour about our own tooling. Our closure oracle — the thing that decides whether a file counts as proved — had **two soundness holes found in a single day**, both by an independent reviewer rather than by the loop, and both now failing closed: one let a candidate print a fake axiom report; the other let a candidate *redefine* `#print axioms` in its own syntax so the check never ran. Neither was visible to a passing test suite. A verification loop is a piece of software, and it earns trust the same way the proofs do.

6 Limitations

These bound the claims, and we would rather state them than have them found.

1. **Different claims per row.** Route A’s row proves a bounded instance; Route B’s tier 2 proves arbitrary N . Only the same-claim pairs of §4.2 compare like with like.
2. **The equivalence is audited, not proved.** Lean-model $\equiv$ TLA⁺-semantics is a hand audit per task. A modelling slip would make the theorem true about the wrong thing.
3. **Model, not code.** Both routes prove properties of models; neither refines to an implementation.
4. **Safety only.** Liveness is out of scope. The published EWD998 figures split on exactly that line (1 person-day invariance, <1 person-day liveness).
5. **No pair was measured at a crossover.** Every N_0 was chosen by the loop’s budget, not by where the two routes trade places — which is *why* §4.2 splits. The cost shape (TLC exponential, proof flat) is measured; the crossover itself is extrapolated.
6. **Small task set.** Four algorithms, eight settled cells. Not a sampled population.
7. **Different precision across tasks.** Three tasks have $R \geq 5$ cells; EWD998’s are single runs, so its figures are existence proofs. Since EWD998 anchors the human comparison, that matters.

8. **Stochastic costs.** Each row is one sample; verdicts are deterministic given an artifact, costs are distributions. Medians carry ranges for that reason.
9. **Cost basis.** Tokens at list price, compute at a stated host rate, researcher time not costed. Both components are reported separately and never merged.
10. **Our own tooling is young,** as §5 records.

7 Conclusion

An AI proof loop closes general theorems for real concurrent algorithms, at a cost of seconds to seventeen minutes and under a millicent, inside a budget two to three orders of magnitude larger than what it uses. **That answers a question TLC cannot be asked** — arbitrary N rather than a bounded instance — and it does so with a mechanically checked artifact rather than a generated one.

It does not make model checking obsolete. On the same bounded claim, TLC is faster on two of four tasks, and it checks the specification’s semantics directly rather than a ported model whose equivalence is a hand audit. **The honest summary is that the two routes answer different questions at different costs, the crossover between them is the quantity that matters, and no experiment yet measures it.** That is the obvious next step: a task calibrated so that both routes are under pressure at the same instance.

References

- [1] K. Konnov, T. Kuppe, and S. Merz. Specification and Verification With the TLA⁺ Trifecta: TLC, Apalache, and TLAPS. In *ISoLA 2022*. <https://members.loria.fr/SMerz/papers/2022-isola.pdf>
- [2] K. Chaudhuri, D. Doligez, L. Lamport, and S. Merz. Verifying Safety Properties With the TLA⁺ Proof System. In *IJCAR 2010*. <https://members.loria.fr/SMerz/papers/ijcar2010.pdf>
- [3] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE-28*, 2021.
- [4] The Mathlib Community. The Lean Mathematical Library. In *CPP 2020*.
- [5] L. Lamport. A New Solution of Dijkstra’s Concurrent Programming Problem. *Communications of the ACM*, 17(8), 1974.
- [6] E. W. Dijkstra. Derivation of a Distributed Termination Detection Algorithm. EWD manuscript 998.
- [7] G. Le Lann. Distributed Systems — Towards a Formal Approach. In *IFIP Congress*, 1977.
- [8] E. Chang and R. Roberts. An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes. *Communications of the ACM*, 22(5), 1979.
- [9] DeepSeek-AI. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. arXiv:2504.21801.
- [10] Numina and Kimi teams. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. arXiv:2504.11354.
- [11] Goedel-LM. Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving. arXiv:2502.07640.